

ESE 499 Undergraduate Research –Summer 2026

Project Report
Line-by-Line Code Analysis

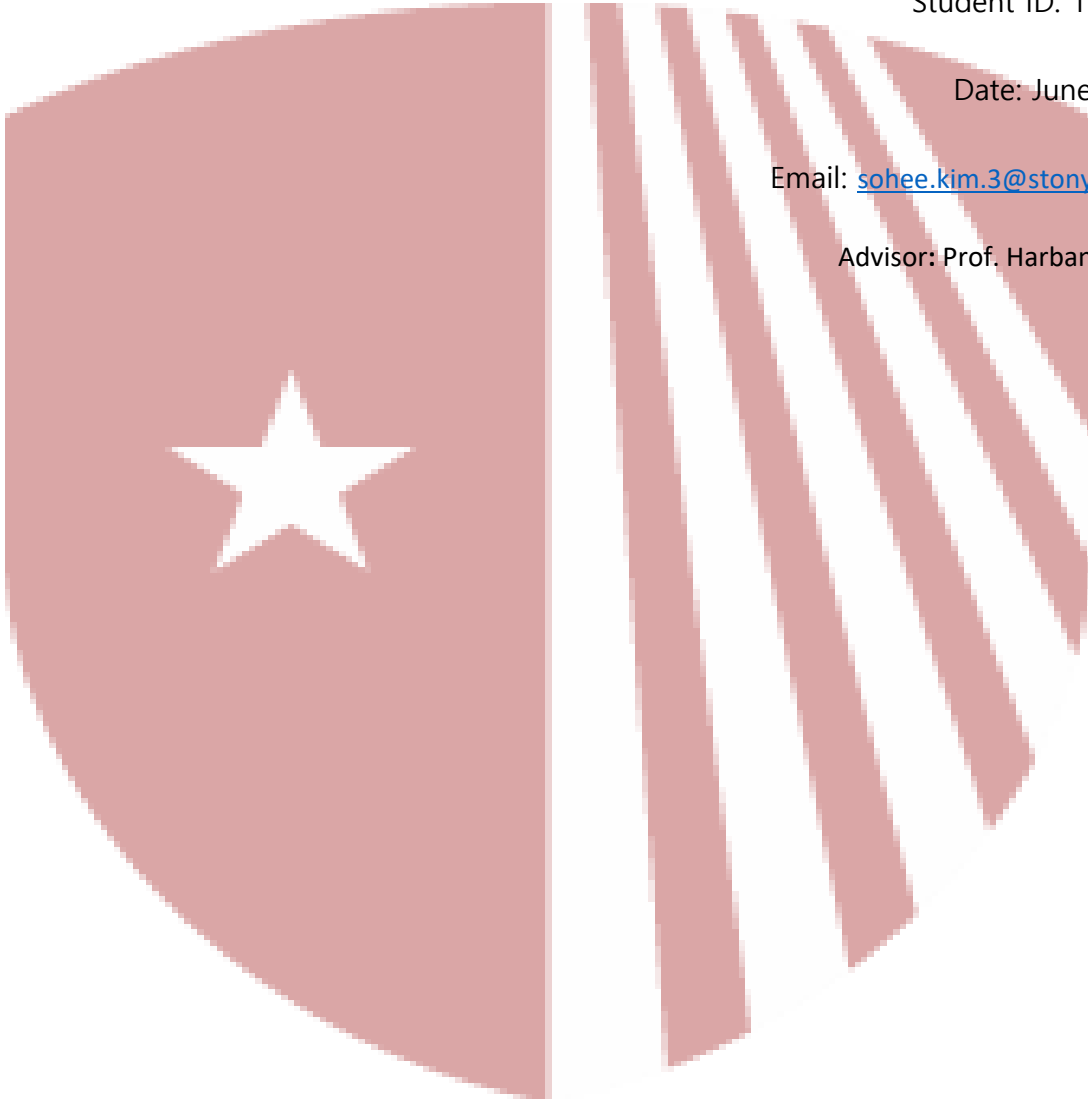
Name: Sohee Kim

Student ID: 117643907

Date: June 26, 2026

Email: sohee.kim.3@stonybrook.edu

Advisor: Prof. Harbans Dhadwal



1. Tx code

```

import time
import numpy as np
import adi
import scipy.signal as signal
from scipy.io import wavfile

# =====
# 1. Load and Pre-process WAV File
# =====
try:
    fs_audio, audio_data = wavfile.read("music.wav")
except FileNotFoundError:
    print("ERROR: 'music.wav' file not found in the current directory!")
    exit(1)

if len(audio_data.shape) > 1:
    audio_data = audio_data[:, 0]

audio_data = audio_data.astype(np.float32)
audio_data /= np.max(np.abs(audio_data))

# =====
# 2. Resampling to Match SDR Sample Rate
# =====
decim_factor = 12
fs_sdr = 44100 * decim_factor # 529,200 Hz

if fs_audio != 44100:
    num_samples = int(len(audio_data) * fs_sdr / fs_audio)
    tx_signal = signal.resample(audio_data, num_samples)
else:
    tx_signal = signal.resample(audio_data, len(audio_data) * decim_factor)

tx_signal = (tx_signal * 5000).astype(np.complex64)

# =====
# 3. Hardware Initialization
# =====
try:
    sdr = adi.Pluto("ip:192.168.2.1")
except Exception as e:
    print(f"Hardware Connection Failed: {e}")

```



```

    exit(1)

sdr.sample_rate = int(fs_sdr)
sdr.tx_rf_bandwidth = int(fs_sdr)
sdr.tx_lo = int(920000000)
sdr.tx_hardwaregain_chan0 = -10

# =====
# 4. Stream Audio over Air
# =====
buffer_size = 16384 * 2
num_chunks = len(tx_signal) // buffer_size
chunk_duration = buffer_size / fs_sdr

print("=====")
print("Streaming 'music.wav' over the air...")
print("Frequency: 920.0 MHz | Buffer Protection Active")
print("=====")

try:
    while True:
        for i in range(num_chunks):
            chunk = tx_signal[i * buffer_size : (i + 1) * buffer_size]
            sdr.tx(chunk)
            time.sleep(chunk_duration * 0.95)

        print("[INFO] Audio playback finished. Looping from the
beginning.")

except KeyboardInterrupt:
    print("\n[INFO] Transmission stopped by user.")

```

Fig 1. Tx full code

[Package & Library Import Section]

- **import time:** A built-in library used to control time. It allows us to pause code execution temporarily.
- **import numpy as np:** A fundamental library used for fast mathematical operations and handling large data arrays (matrices) efficiently.
- **import adi:** The dedicated hardware driver package from Analog Devices that allows Python code to interface with and control the PlutoSDR.
- **import scipy.signal as signal:** A digital signal processing (DSP) library used for changing sampling rates, filtering, and frequency modulation.

- **from scipy.io import wavfile:** A specialized file I/O utility used to read and write standard .wav audio files directly within Python.

[Section 1: Loading and Pre-processing the Audio File]

- **try:** A safety mechanism that tells Python to "try running the following block, and if an error occurs, handle it gracefully instead of crashing."
- **fs_audio, audio_data = wavfile.read("music.wav"):** Reads the file music.wav and extracts its native sampling rate (fs_audio) and raw sound amplitude values (audio_data) into memory arrays.
- **except FileNotFoundError::** The backup block that runs immediately if the file music.wav is missing from the working directory.
- **print("ERROR: ...") / exit(1):** Prints a clear warning log to the console and safely terminates the script so it doesn't try to process an empty file.
- **if len(audio_data.shape) > 1::** A conditional check that detects if the source audio is a multi-channel (stereo) file with distinct left and right tracks.
- **audio_data = audio_data[:, 0]:** If it is a stereo file, this extracts only the first channel (left channel), converting it into a single-channel 'mono' signal. (SDR hardware uses a single antenna interface and natively transmits a single-channel payload).
- **audio_data = audio_data.astype(np.float32):** Casts the raw audio integers into high-precision floating-point numbers (float32), which are necessary for precise DSP calculations.
- **audio_data /= np.max(np.abs(audio_data)):** Peak Amplitude Normalization. It divides every value in the audio array by its absolute maximum value. This maps the signal amplitude strictly between -1.0 and 1.0, preventing digital clipping and audio distortion.

[Section 2: Resampling to Match the SDR Sample Rate]

- **decim_factor = 12:** Defines a static scaling multiplier (12x interpolation) shared with the receiver.
- **fs_sdr = 44100 * decim_factor:** Calculates the target high-speed sampling rate (529,200Hz) required by the PlutoSDR hardware to transmit the radio waves smoothly.
- **if fs_audio != 44100::** Checks whether the original file's sample rate deviates from the standard CD quality rate of 44.1kHz.
- **num_samples = int(...) / tx_signal = signal.resample(...):** If the original rate is non-standard, this calculates the new sample size and mathematically resamples the audio vectors up to the target rate.
- **else: / tx_signal = signal.resample(..., len(audio_data) * decim_factor):** If the file is standard 44.1kHz, it executes an exact 12x interpolation, filling the gaps between points mathematically to boost the sample count.

- **tx_signal = (tx_signal * 5000).astype(np.complex64):** SDR hardware requires complex inputs (IQ data pairs). This line maps our real audio data into a complex number format (complex64) and amplifies the data by a factor of 5000 so the physical antenna can radiate it across the room.

[Section 3: Hardware Initialization & Configuration]

- **try::** A protective block that keeps the program from crashing if the PlutoSDR is unplugged or fails to connect over the network layer.
- **sdr = adi.Pluto("ip:192.168.2.1"):** Directs Python to connect to the PlutoSDR's static network address (192.168.2.1), instantiating the hardware handle object named sdr.
- **except Exception as e: / print(...) / exit(1):** If the device cannot be resolved, this prints the exact hardware exception and safely aborts execution.
- **sdr.sample_rate = int(fs_sdr):** Hardcodes the physical DAC (Digital-to-Analog Converter) sampling rate inside the PlutoSDR to our calculated 529,200Hz.
- **sdr.tx_rf_bandwidth = int(fs_sdr):** Matches the RF analog bandwidth to our sampling rate, ensuring that the transmitted spectrum profile does not get clipped or filtered out by the radio front-end.
- **sdr.tx_lo = int(920000000):** Sets the local oscillator (LO) carrier frequency to 920.0MHz. This is the physical channel over which the radio waves travel, and the receiver must tune to this exact frequency.
- **sdr.tx_hardwaregain_chan0 = -10:** Lowers the antenna output power by 10dB from its maximum capacity. This back-off margin prevents the close-range receiver antenna from getting saturated and clipping the incoming signal.

[Section 4: Real-time Over-the-Air Streaming Loop]

- **buffer_size = 16384 * 2:** Specifies the optimal memory block size (32,768 continuous samples) to pass to the hardware in a single burst.
- **num_chunks = len(tx_signal) // buffer_size:** Divides the total upsampled audio vector by the chunk size to calculate exactly how many slices the program needs to send.
- **chunk_duration = buffer_size / fs_sdr:** Calculates the precise physical duration (in seconds) it takes for a single 32,768-sample block to be fully radiated out by the antenna.
- **print(...):** Outputs an informative status block to the console indicating that streaming is active.
- **try::** Monitors the loop for user interruptions, allowing a clean exit when the script is stopped.
- **while True::** An infinite runtime loop that keeps the audio streaming continuously without halting.

- **for i in range(num_chunks)::** Iterates through the pre-calculated audio slices one chunk at a time.
- **chunk = tx_signal[...]:** Slices the current 32,768-sample segment from the global complex audio array.
- **sdr.tx(chunk):** [Core Functional Call] Transfers the sliced data block directly into the PlutoSDR's physical DMA buffer, triggering immediate 920MHz RF radiation over the air.
- **time.sleep(chunk_duration * 0.95):** [Critical Timing Correction] Pauses the Python script execution for exactly 95% of the physical chunk transmission time. This gives the PlutoSDR hardware sufficient breathing room to broadcast the contents of its buffer before the next block is pushed, completely resolving the hardware-software race condition that previously caused Broken Pipe errors.
- **print("[INFO] ..."):** Once the last chunk is sent, it prints a loop notification to the terminal, and the while True parent constraint immediately restarts transmission from the very first audio chunk.
- **except KeyboardInterrupt: / print(...):** Catches a manual user exit command (Ctrl + C), prints a graceful cleanup message, and closes the hardware sessions smoothly.

2. Rx code

```
import numpy as np
import adi
import sounddevice as sd
from scipy.signal import decimate

# =====
# 1. System Configuration
# =====
fs_audio = 44100
decim_factor = 12
fs_sdr = fs_audio * decim_factor # 529,200 Hz
center_freq = 920000000          # 920.0 MHz

# =====
# 2. Hardware Initialization (USBIPD Context)
# =====
try:
    sdr = adi.Pluto("ip:192.168.2.1")
except Exception:
    sdr = adi.Pluto()

sdr.sample_rate = int(fs_sdr)
```

```

sdr.rx_rf_bandwidth = int(fs_sdr)
sdr.rx_lo = int(center_freq)
sdr.rx_buffer_size = 12288
sdr.gain_control_mode_chan0 = "manual"
sdr.rx_hardwaregain_chan0 = 20

# =====
# 3. Real-time Audio Callback Function
# =====
def audio_callback(outdata, frames, time_info, status):
    try:
        rx_data = sdr.rx()
        rx_data = rx_data - np.mean(rx_data)

        demod_signal = np.angle(rx_data[1:] * np.conj(rx_data[:-1]))

        audio_out = decimate(demod_signal, decim_factor, zero_phase=False)

        max_val = np.max(np.abs(audio_out))
        if max_val > 0:
            audio_out = audio_out / max_val * 0.3

        outdata[:, 0] = audio_out[:frames].astype(np.float32)
    except Exception:
        outdata.fill(0)

# =====
# 4. Stream Audio to Windows Host via WSL2
# =====
stream = sd.OutputStream(samplerate=fs_audio, channels=1,
callback=audio_callback, blocksize=1024)
stream.start()

print("=====")
print(f"WSL2 Real-time FM Receiver Active! Listening at {center_freq /
1e6} MHz...")
print("Press Ctrl+C to exit.")
print("=====")

try:
    while True:
        sd.sleep(1000)
except KeyboardInterrupt:
    print("\n[INFO] Closing audio stream and cleaning up hardware links.")

```



```
stream.stop()
stream.close()
```

Fig 2. Rx full code

[Package Imports & System Configuration]

- **import numpy as np:** A fundamental library used to perform high-speed matrix calculations on large blocks of wireless IQ samples.
- **import adi:** The native driver package from Analog Devices used to interface with the PlutoSDR and capture raw over-the-air RF signals.
- **import sounddevice as sd:** An audio interface tool used to push baseband audio vectors directly to the host computer's speakers in real time.
- **from scipy.signal import decimate:** A specialized DSP module that downsamples high-frequency vectors while applying anti-aliasing low-pass filters to prevent distortion.
- **fs_audio = 44100:** Sets the standard CD-quality audio playback sample rate (44.1kHz) for the final speaker output.
- **decim_factor = 12:** Defines the exact downsampling multiplier (12x decimation) matching the transmitter's scaling.
- **fs_sdr = fs_audio * decim_factor:** Computes the high-speed over-the-air RF sampling rate (529,200Hz) for the PlutoSDR's physical analog-to-digital converter (ADC).
- **center_freq = 920000000:** Establishes the operational carrier frequency to 920.0MHz, which is the channel the receiver monitors.

[Hardware Instantiation & Optimization]

- **try: sdr = adi.Pluto("ip:192.168.2.1"):** Attempts to route the hardware handle directly over the fixed TCP/IP socket network to bypass virtual network limits in WSL2.
- **except Exception: sdr = adi.Pluto():** Provides a local driver fallback instantiation path if the direct IP binding fails.
- **sdr.sample_rate = int(fs_sdr):** Programs the receiver hardware clock speed to match our target rate of 529,200Hz.
- **sdr.rx_rf_bandwidth = int(fs_sdr):** Matches the analog frontend filtering width to the sample rate to capture the complete transmitted spectrum without signal truncation.
- **sdr.rx_lo = int(center_freq):** Tunes the local oscillator inside the PlutoSDR to the exact 920.0MHz frequency of the transmitter.
- **sdr.rx_buffer_size = 12288:** Specifies the optimal complex sample block size allocated for each data fetch operation.
- **sdr.gain_control_mode_chan0 = "manual":** Enforces a strict manual gain control loop to lock the amplification factors and prevent floating gain fluctuations.



- **sdr.rx_hardwaregain_chan0 = 20:** Boosts the incoming weak over-the-air RF signal by 20dB at the hardware stage to optimize algorithmic processing conditions.

[Asynchronous Real-time Audio Callback Loop]

- **def audio_callback(outdata, frames, time_info, status)::** An asynchronous background thread function automatically fired by the OS audio device driver whenever the hardware buffer demands new samples.
- **rx_data = sdr.rx():** Fetches raw, unmodulated high-frequency complex IQ vectors directly from the PlutoSDR.
- **rx_data = rx_data - np.mean(rx_data):** Removes the DC offset component by subtracting the arithmetic mean of the vector array, centering the signal around 0V.
- **demod_signal = np.angle(rx_data[1:] * np.conj(rx_data[:-1])):** [FM Demodulation Core] Multiplies the current complex sample by the conjugate of the previous sample to calculate the discrete phase derivative, translating frequency changes back into original audio amplitude vectors.
- **audio_out = decimate(demod_signal, decim_factor, zero_phase=False):** Reduces the sampling rate from the high-speed SDR frequency (529.2kHz) down to the audio playback frequency (44.1kHz) via a 12x decimation filter pipeline.
- **max_val = np.max(np.abs(audio_out)):** Measures the maximum absolute amplitude scale within the processed audio chunk array.
- **if max_val > 0: audio_out = audio_out / max_val * 0.3:** Executes peak audio normalization and applies a strict scaling factor of 0.3 to protect the user's hearing and speakers from transient high-volume spikes.
- **outdata[:, 0] = audio_out[:frames].astype(np.float32):** Maps the final, cleanly demodulated and filtered audio stream block directly into the primary hardware speaker output channel buffer.
- **except Exception: outdata.fill(0):** Catches dropouts or driver timing anomalies, filling the buffer block with absolute silence (zeros) instead of outputting loud audio artifacts.

[Runtime Stream Activation & Keep-Alive Loop]

- **stream = sd.OutputStream(...):** Hooks the audio output channel to the system sound driver and registers the underlying callback thread logic.
- **stream.start():** Fires up the asynchronous audio streaming pipeline, initiating real-time over-the-air sound reproduction.
- **print(...):** Logs a clean initialization banner to the console terminal signifying the system is listening.

- **while True: sd.sleep(1000):** A persistent foreground sleep trap that locks the parent process script open, allowing the asynchronous worker callback threads to stream indefinitely.
- **except KeyboardInterrupt: ...:** Intercepts a standard manual termination keypress (Ctrl + C), safely closes the audio hardware pipeline sessions, and triggers a clean script exit.